Test and Validate AI Code You Can Trust with GitHub Copilot

Step 1 – Open Repo

We have prepared a set of demo and timed activities (labs) for you.

git clone QACS-TL/QATVAICCT-M: Test and Validate AI Code You Can Trust

Step 2 – AI generated CSV Import Service

Time

20 Minutes

Purpose

Primarily focuses on:

- Correctness
- Completeness
- Requirement validation

Scenario

A team has used GitHub Copilot to create a CSV importer used to load customers into a CRM system.

- The code has been tested.
- All tests pass.

You have been asked to verify whether the solution is correct, complete and trustworthy.

Learner Tasks

Task 1

From Visual Studio Code, open the C:\QATVAICCT-M\Lab Repos\Timed Activity 1\Starter folder

Review the requirements in the Requirements.md file

Review the code in the customer_importer and customer_importer_test modules

Task 2

Open a terminal window in GitHub Copilot (Select "New Terminal" from the Terminal menu)

Run the tests by entering:

python -m pytest customer_importer_test.py

Everything passes.

Task 3

Use Copilot Chat

Prompt:

Review the implementation, tests and requirements. Identify behaviours that are not currently tested.

Task 4

Generate additional tests.

Prompt:

Generate pytest tests for:

- duplicates
- malformed data
- invalid emails
- missing fields
- large files
- error handling

Run the tests (as you did in Task 2) and ensure the new tests fail.

Task 5

Update implementation and rerun the tests to ensure they all pass.

Task 6

Determine:

- Can we trust this importer?
- What evidence supports that view?

Step 3 – AI generated API rate limiter

Time

20 Minutes

Purpose

Primarily focuses on:

- Fitness for Purpose
- Reliability
- Security
- Independent Validation

Scenario

A team created an API rate limiter using GitHub Copilot.

The feature passed testing and is ready for release.

You must determine whether it is fit for production.

Learner Tasks

Task 1

Using Visual Studio Code open the Lab Repos/Timed Activity 2/Starter folder

Review implementation and note any issues you can see with the code as it stands

Open a terminal window in VSC (Select "New Terminal" from the Terminal menu)

Run the tests to ensure they all pass:

python -m pytest rate_limiter_test.py, test_main.py

Task 2

Use GitHub Copilot.

Prompt:

Act as a senior software architect. What production problems could occur with this implementation?

Task 3

Security Review

Prompt:

Act as an API security reviewer. Identify security, availability and abuse risks.

Task4

Generate production-grade tests.

Prompt:

Generate pytest tests that exercise reliability, concurrency and failure scenarios.

Run the tests and ensure the newly generated ones fail. Note AI may generate a additional test file if it does then add it to the comma separated list:

python -m pytest rate_limiter_test.py, test_main.py, test_?????.py

Task 5

Implement code.

Prompt:

Implement the code so that all the tests pass

Task 6

Classify findings.

For each issue determine whether it affects:

- Correctness
- Completeness
- Fitness for Purpose

Task 7

Recommend production readiness.